

Developed Algorithm for Making Up Convex Hull Based on Binary Tree

Naing Linn Aung

Department of Informatics and Computer
Software System, National Research
University of Electronic Technology (MIET))
Zelenograd, Moscow, RUSSIA
nainglinnaung53@gmail.com

Evgeni M. Portnov

Department of Informatics and Computer
Software System, National Research
University of Electronic Technology (MIET))
Zelenograd, Moscow, RUSSIA
evgen_uis@mail.ru

Kirill O. Epishin

Department of Informatics and Computer
Software System, National Research
University of Electronic Technology (MIET))
Zelenograd, Moscow, RUSSIA
evgen_uis@mail.ru

Abstract—In computer vision, convex hulls are used for pattern recognition and other image analysis using special points. This paper suggests the developed algorithm for making up a convex hull based on a binary tree. The main idea of the algorithm, proposed in this paper, is very similar to the underlying principle incremental algorithm. The new algorithm quite quickly exceeds the running time of the Graham algorithm with a large number of points in the convex hull. This is due to the best complexity of the algorithm. The Graham's algorithm has $O(n \log n)$ complexity when the new algorithm has $O(n \log h)$ average complexity.

Keywords—convex hulls, descriptors, binary tree, image, object, points

I. INTRODUCTION

In computer graphics, convex hulls can be used to detect collisions of objects [1]. The field of computational geometry related to the construction of convex hulls has a lot of scientific research. A huge number of algorithms, applications, and techniques have been developed based on this task. However, this does not mean that the existing algorithms are perfect. Speed of operation is one of the most important characteristics of the algorithm, especially for computer vision algorithms, in particular, finding the object descriptor in the image. This work was primarily done to improve this characteristic [2]. Algorithms that have an optimal complexity of $O(n \log h)$ have a too large constant and therefore not used in practice. All computer vision libraries used algorithms with the complexity of $O(n \log n)$, where n is the number of points in the original set of points, and h is the number of points in the final convex hull. Such algorithms will lose in speed at a small number of points in the convex hull.

The Graham algorithm and other algorithms are used complexity $O(n \log n)$. Although they are not optimal, but the fastest. On the other hand, there is an optimal algorithm for complexity that is not used, and on the other hand, an algorithm that is not optimal, but is actively used in practice.

The asymptotics of the number of vertices of the convex hull for n points clearly shows that a small number of points prevail in the final convex hull:

- with a uniform distribution inside the circle – $\theta(n^{1/3})$;
- with a uniform distribution inside a fixed convex polygon – $\theta(\log n)$;

- with a two-dimensional normal distribution – $\theta((\log n)^{1/2})$ [3].

So, a small set of points in the final convex hull has a working time that can be improved.

II. FORMALIZATION OF THE CONCEPT OF A CONVEX HULL

To develop an algorithm for constructing a convex hull of a finite set of points on a plane, it is necessary to formalize this concept.

Let S be a set of points. The convex hull of a given set is the smallest convex set containing S . A formally convex hull can be defined as the intersection of all convex sets that contain S . This definition is the most popular and general, but it is too far from practice, so we will use another. Note that the developed algorithm should work only for the plane, so we will use a definition that works only for the case of two-dimensional space.

To describe the concept of a convex hull, it is necessary to introduce the predicate $ccw(A, B, C)$, which will also be used to describe algorithms.

$$\det(A, B, C) = \begin{vmatrix} x_A & y_A & 1 \\ x_B & y_B & 1 \\ x_C & y_C & 1 \end{vmatrix}, \quad (1)$$

$$ccw(A, B, C) = \det(A, B, C) > 0 \quad (2)$$

To formalize the concept of a convex hull, we again need the predicate ccw described in formula (2). The convex hull of a set S is the convex polygon T with vertices $t_i \in S$, numbered counterclockwise for $i = 1, \dots, n$ [4]. So the condition (3) is described as follow:

$$\forall [t_i, t_{i+1}] \in T, p \in S \setminus \{t_i, t_{i+1}\} : ccw(t_i, t_{i+1}, p). \quad (3)$$

This expression states that every point p belonging to S except the current one ribs t_i, t_{i+1} , is located to the left of it. This property of the convex hull is perfectly shown in Fig.1 using oriented edges. It can be seen that there are no points to the

right of the edge, otherwise, this edge would not belong to the convex hull.

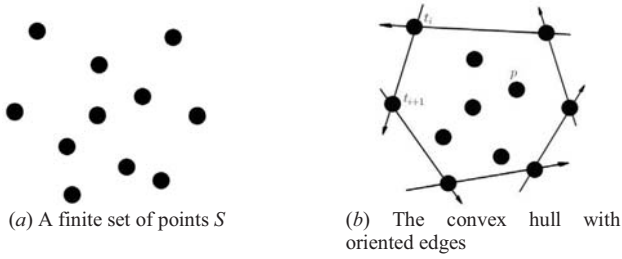


Fig. 1. Formalization of the concept of a convex hull

Note that this definition does not violate the requirement of the minimality of the selected polygon since we compose a convex hull of points that belong to the set S .

The main idea of the algorithm proposed in this paper is very similar to the underlying principle incremental algorithm [5,6]. We can store points in a height-balanced binary search tree. The points in the incremental algorithm were stored in the upper and lower trees and were sorted by the x coordinate. We will use a different approach, which will allow you to store points in only one tree, nodes (points) will be sorted by an angle relative to the approximate center of the convex hull. To explain the operation of the algorithm, we need the predicate ccw already known to us, defined in formula (2) and the concept of an extremal point. The extreme point of the set S is the point that exactly belongs to the convex hull of S . Usually it is the point that lies farthest from all by some parameter. For example, the leftmost or highest point will fit into the resulting convex hull.

Let a set of points S be given and it is necessary to find the convex hull of this set.

The first step of the algorithm is to find the approximate center that will be used to sort the points in the tree. A good approximation of the center for our purposes is the arithmetic mean of the three extreme points:

- the left extreme point A ;
- right extreme point B ;
- point C , which is farthest from line A, B .

Fig. 2a shows an example of the extreme points found in the first step of the algorithm. Then, in fig. 2b, point D is the approximate center of the convex hull we were looking for. After finding the center, we add points A, B, C to our search tree. The remaining points will be added at random. Consider the process of adding another point to the current convex hull of a set. Suppose we have already built a search tree that contains our convex hull sorted by an angle relative to the center D found earlier. Now we need to add a new point A .

The first thing to find is the two points L and R . The points L, R are the points to the left and the right of A , respectively, in the angle relative to the center D . As can be seen in Fig. 3, all points are stored in an angle relative to point D , and therefore searching for points is not difficult.

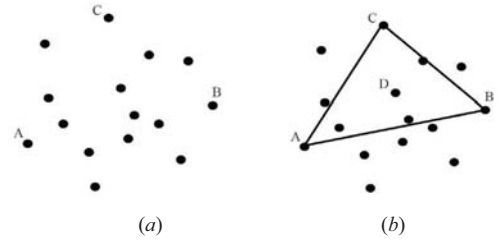


Fig. 2. Finding the approximate center of the convex hull

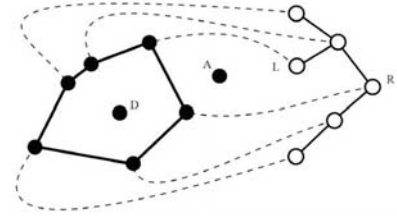


Fig. 3. Finding the points L, R in the binary search tree

Note that it may turn out that the points L and R are in opposite parts of the binary search tree. This is shown in fig. 4. Therefore, we will assume that our used balanced binary search tree is cyclic. Note that this does not require any special additional data structures. All we need to do is to go around the tree and reach the last point, go to the first.

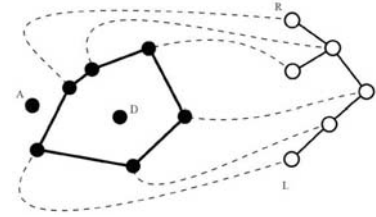


Fig. 4. Demonstration of the cyclicity of the used search tree.

After that, there can be 2 cases. If $ccw(L, R, A)$ is not satisfied, then the new point A should not be part of the convex hull and may be skipped. This case is shown in Fig. 5a. Otherwise, A must be part of the convex hull and we will continue to add it. This is shown in Fig. 5b.

So, we have determined that point A must be on a convex hull. Now we need to remove the points that will be inside the shell after adding point A . This is done by iterating over the points to the right (a similar algorithm should be performed for the left side) from the new point. To determine whether a point should be deleted or not, we will use an approach similar to the one used in the Graham algorithm. Every two adjacent edges of the convex hull must lie counterclockwise.

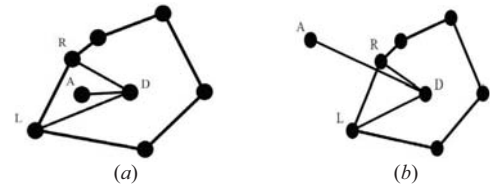


Fig. 5. Adding a new point to the convex hull

Let the next point that is considered for removal be B . Consider the process of deleting point B .

1. Name the point that comes next after B , C .
2. If $ccw(C, B, A)$ then B should not be deleted and we must stop deleting points in this direction because all points after B already satisfy the convex hull condition. This predicate also checks the condition for lying adjacent edges counterclockwise, which was mentioned earlier.
3. Otherwise, point B must be removed, since it is inside the new convex hull.
4. Then we need to consider point C as the next candidate for removal and go to step 1.

In Fig. 6a and 6b, we can see that points C, B, A lie clockwise, so point B must be removed from the convex hull. Fig. 7a demonstrates that we must stop deleting points to the right of the new point, since C, B, A now forms a triple that lies counterclockwise. Similarly, the deletion does not need to be done to the left of the point, as shown in Fig. 7b. Indeed, the figures show that in the end, we got a ready-made convex hull of the points added at the moment.

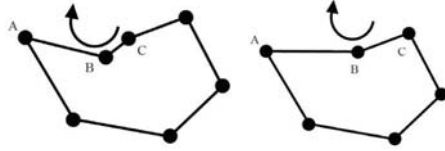


Fig. 6. Removing points that are inside the convex hull

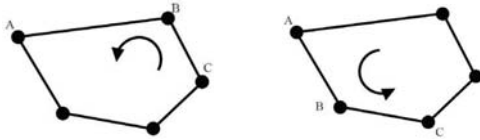


Fig. 7. The criterion for completing removing points

This process is very similar to the process of adding points in the incremental algorithm. There are such small differences as one tree instead of two, the cyclical nature of this tree, the absence of a boundary case at the junction of two trees, etc. Although they are small, they allow the algorithm to quickly add points to the convex hull. A block diagram of the entire algorithm is shown in Fig. 8 and 9.

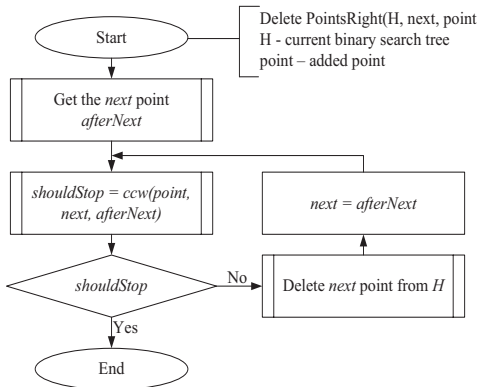


Fig. 8. The block diagram of the delete points function before adding a point

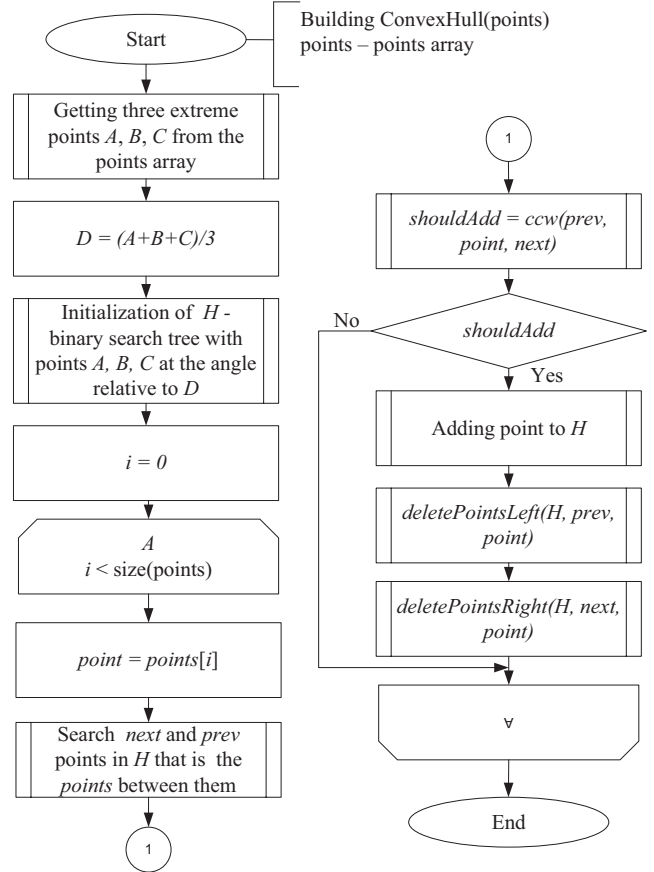


Fig. 9. The block diagram of the proposed algorithm

III. IMPLEMENTATION DETAILS

The first important detail that simplifies the implementation of our algorithm is the transformation of points. It is much more convenient to assume that the approximate center D , the receipt of which was described in the previous chapter, is point $(0, 0)$. As shown in equation (2), we calculate ccw using det , and it is easier to calculate $det(A, B, D)$ if $D_x = 0$ and $D_y = 0$. We also must not forget about the final goal of computing the convex hull. To calculate the descriptor, it is again much more convenient to use the coordinates already transferred to the center of the object, since the position of the object in the image should not affect the descriptor in any way.

The second detail, which is also important in the implementation, is the angles relative to the center. Calculation of angles through trigonometric operations is a very long time, and in this case, our algorithm could not compete in performance with the most popular convex hull calculation algorithms.

It is necessary to come up with a predicate that would determine the position of points in a tree that stores the current convex hull. The first attempt to come up with such a predicate will be using the ccw predicate, as shown in (4).

$$A < B = ccw(A, B, (0, 0)) \quad (4)$$

Note that we can use the point $(0, 0)$ as a center, since points A, B are already transformed into a new coordinate system.

Unfortunately, such a predicate does not correspond to strict weak ordering [7,8,9], because transitivity is not satisfied. As can be seen in Fig. 10a, $ccw(A, B, D)$ is executed, therefore A is less than B . At the same time, $ccw(B, C, D)$ is executed, therefore B is less than C . But Fig. 10b shows at the same time that $ccw(A, C, D)$ does not hold, that is, A is not less than C . As a result, we have $A < B < C \leq A$, which is wrong.

So, we need to make a function that takes points A and B and returns true if A is less than B , otherwise false. This function will be used in the balanced search tree as a comparator. The following algorithm was used:

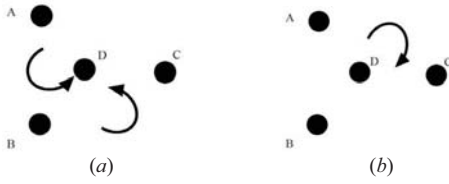


Fig. 10. Nontransitive predicate

Algorithm BSTPredicate – comparator for comparing points

```

1: procedure BSTPredicate ( $A, B$ )
2:    $leftUp \leftarrow 0 < y_A$ 
3:    $rightUp \leftarrow 0 < y_B$ 
4:   if  $leftUp \neq rightUp$  then return  $leftUp < rightUp$ 
5:   end if
6:   if  $(y_A = 0) \& (y_B = 0)$  then return  $(0 < x_A) < (0 < x_B)$ 
7:   end if return  $ccw(A, B, (0, 0))$ 
8: end procedure

```

An example of the point order calculated using the BSTPredicate algorithm is shown in Fig. 11.

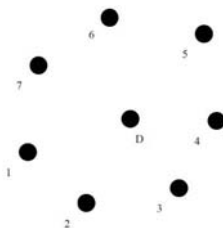


Fig. 11. Sort predicate points BSTPredicate

This simple function implements the correct predicate for sorting. To do this, initially, it divides the entire coordinate plane into two parts: upper and lower. As we see the predicate takes 2 points A and B . Consider three cases:

1. If these points lie on different sides. For example, point A lies below point B , then the predicate will return true, which means that A is less than B .

2. The boundary case, when both points are on the x -axis. Then we need to determine which side of the y -axis they are. If

with one, then the currents are equal because the angle to them is the same. Otherwise, there is a smaller point that is on the left.

3. If the points are in the same half-plane, then the predicate ccw , shown in formula (2), does its job perfectly.

IV. DIFFICULTY RATING ALGORITHM

Given a set S of size N point, the first is necessary to determine - the time complexity.

At the initialization step of the algorithm, three extreme points are searched. This is done in $2N$ steps since we must first find the left and right points and then find the other point farthest from them. Then the points are considered in random order. Initially, we need to find the position of the point in the binary tree. This operation is done in $O(\log h)$, where h is the current number of points in the convex hull.

If this point needs to be added, then all points are deleted that, after adding the point, will appear inside the convex hull. The complexity of the delete point operation can vary for different balanced binary search trees. The C++ language standard says that the amortized complexity of the delete operation from $std::set$ by the iterator is $O(1)$ [10]. Therefore, we assume that the delete operation has constant complexity.

During the operation of the algorithm, the maximum number of deletions cannot exceed N , since only N points are considered. Therefore, this operation takes $O(N)$ time.

Let the maximum size of the convex hull during the operation of the algorithm be M . Then the complexity of finding points for the entire duration of the algorithm is $O(N \log M)$.

As a result, the complexity of the algorithm is:

$$O(2N + N + N \log M) = O(N \log M) \quad (5)$$

Note that in the usual case, the number of points in the tree will be approximately H for the entire time the algorithm is running, which allows us to say that the average complexity will be $O(N \log H)$, where H is the number of points in the final convex hull. Note that since points are added at random, it is usually this complexity that will be achieved. If we consider the points, not in random order, then it is possible to select tests for which the complexity of the algorithm will be $O(N \log N)$. Therefore, we believe that in the worst case, our algorithm will have just such a time complexity.

In addition to storing the input data, the algorithm proposed in this paper also needs to store a balanced binary search tree with points that now belong to the convex hull. Therefore, the memory complexity will be $O(M)$. Like time complexity, the average is $O(H)$, when in the worst case it is $O(N)$. For comparative testing, the CGAL library was chosen as one of the most popular libraries for computational geometry. This library provides several functions for constructing a convex hull [11,12,13]:

- *ch_aki_toussaint* — a function using the AkiToussaint algorithm [14], the complexity is $O(n \log n)$;
- *ch_bykat* — a function using the Eddy algorithm [15], the complexity is $O(nh)$;
- *ch_bykat* — a function using a non-recursive version of the Eddy algorithm [16], the complexity is $O(nh)$;
- *ch_graham_andrew* — a function using the Andrew version of Graham's algorithm [17], the complexity is $O(n \log n)$;
- *ch_jarvis* — a function using the Jarvis algorithm [18], the complexity is $O(nh)$.

As we can see, only 2 algorithms have complexity $O(n \log h)$. We will conduct a comparison with the Graham algorithm as the most popular of the proposed ones.

The following are the parameters of the system on which the comparison was carried out:

1. Processor: Intel Core i5-6600K
2. RAM: Kingston HyperX FURY Black DDR4 2666MHz 8GB
3. Operating system: Windows 10 (Version 1803)
4. Compiler: Microsoft (R) C / C ++ Optimizing Compiler Version 19.11.25547 for x86

Let see the comparison works when comparing our algorithm with the Graham algorithm.

The result of comparing the algorithms for generating points uniformly distributed inside the unit circle is shown in Fig. 12. With a two-dimensional distribution of points shown in Fig. 13. Looking at these graphs, we can see that the proposed algorithm is much better than the Graham algorithm in all cases, but this is certainly not the case.

One way to test the algorithm using a new technique is to fix the number of points in the original set and change the percentage of points that are in the convex hull. If we do this, we will get Fig. 14, 15 for 10 and 50 thousand points, respectively.

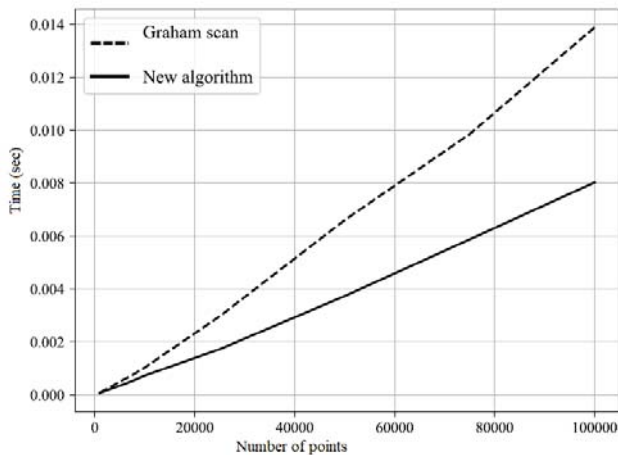


Fig. 12. Comparison of algorithms with a uniform distribution within a circle

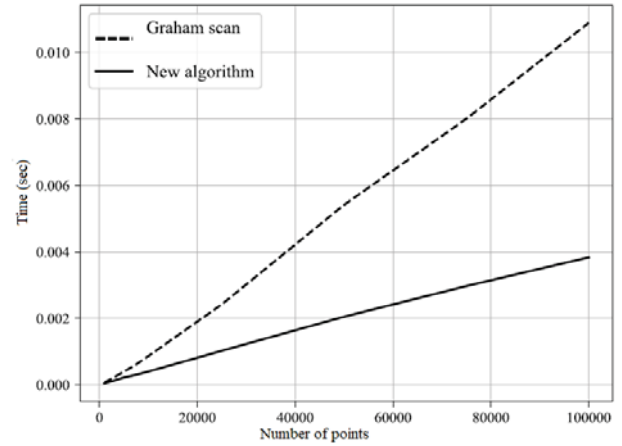


Fig. 13. Comparison of algorithms with a two-dimensional normal distribution of points

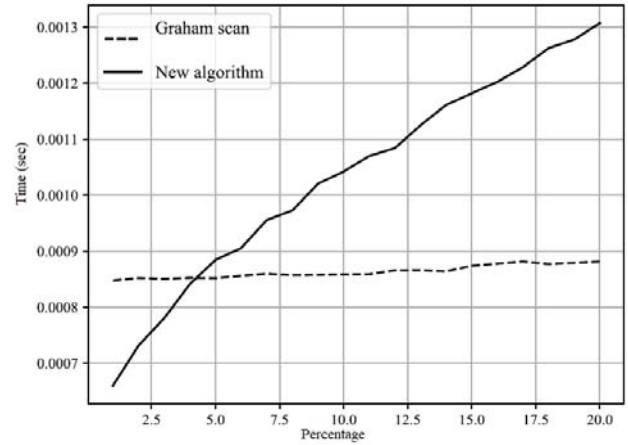


Fig. 14. Comparison of algorithms for $n = 10000$

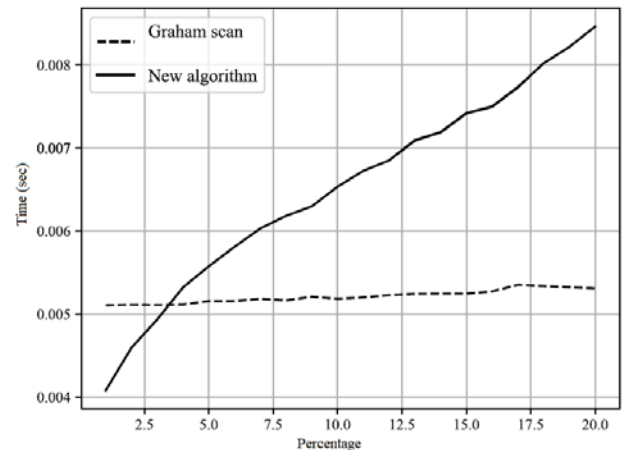


Fig. 15. Comparison of algorithms for $n = 50000$

It is easy to see that the algorithm proposed in this work is faster than the Graham algorithm only at a small percentage of points. This is due to the best complexity of the algorithm. Graham's algorithm has $O(n \log n)$ complexity when the new

algorithm has $O(n \log h)$ average complexity. Such a difference gives gain on small h . Nevertheless, the comparison result in the graphs is not representative, because the comparison according to the new method should be carried out according to two parameters – n (the number of points in the initial set) and h (the number of points in the convex hull). Table 1 shows such a comparison.

TABLE I. THE RATIO OF THE RUNNING TIME OF THE NEW ALGORITHM TO THE RUNNING TIME OF THE GRAHAM ALGORITHM

Number of points	Percentage of points in the convex hull					
	1	3	4	6	8	10
1000	-17.03	-0.08	-3.99	2.94	10.69	16.2
2500	-23.19	-7.71	-7.24	-0.08	9.13	14.4
5000	-19.86	-12.23	-1.77	2.45	11.27	17.79
7500	-23.57	-8.26	-1.92	6.92	8.31	17.85
10000	-22.14	-8.25	-1.32	5.81	13.49	21.39
25000	-19.54	-2.22	3.72	12.42	21.92	27.96
50000	-20.10	-3.41	4.03	12.64	19.69	26.03
75000	-17.74	-0.11	5.25	16.64	23.07	31.15
100000	-15.61	2.59	9.45	19.6	27.81	41.18

The values of the table are the ratio of the running time of the new algorithm to the running time of the Graham algorithm translated in percent. For convenience, we have replaced the direct use of h as the number of points with the percentage of points in the original set that will go into the convex hull.

Note that the new algorithm pretty quickly exceeds the running time of the Graham algorithm with a large number of points in the convex hull. The problem is that Graham's algorithm carries out all its operations on the data lying in the array, which allows excellent use of the processor cache [19, 20]. At the same time, the data structure that underlies our algorithm is a red-black tree. This is a balanced binary search tree that stores its data randomly in memory.

Therefore, when iterating over this data structure, we lose a lot of time. One can try to improve the algorithm proposed in the work by replacing the underlying data structure with any other balanced binary search tree. Or even not to a tree, but as a whole to a structure that has the necessary operation algorithms for operation.

V. CONCLUSION

The algorithm for constructing a convex hull proposed in this paper has the right to exist and can compete with other algorithms on a certain set of problems. The developed algorithm can be improved and optimized for a specific class of tasks by changing the basic data structure on which it works. The classical methodology for evaluating the performance of algorithms does not work as well as applied to the problems of constructing a convex hull since the running time of the

algorithms for this particular task is highly dependent on the output. A new methodology for evaluating performance was proposed.

ACKNOWLEDGMENT

This work was financially supported by the Ministry of Science and Higher Education of the Russian Federation (Unique identifier of the project RFMEFI60519X0181).

REFERENCES

- [1] C. Braune, "Obtaining shape descriptors from a concave hull-based clustering algorithm," Int. Symposium on Intelligent Data Analysis, pp. 61–72, 2016.
- [2] M. Sonka, Image processing, analysis, and machine vision. Cengage Learning, 2014.
- [3] Qhull, 1995. [Online]. Available: <http://www.qhull.org/>
- [4] D. Picardie and Y. Bertot, "Formalizing convex hull algorithms," Int. Conf. on Theorem Proving in Higher Order Logics, pp. 346–361, 2001.
- [5] D. Mitrokhin and R. Golovanov, "Convex hull calculation approach based on BST," Young Researchers in Electrical and Electronic Engineering, pp. 1549–1552, 2018.
- [6] M. De Berg, Computational geometry. Springer, 2000.
- [7] R. Osada, "Shape distributions," ACM Transactions on Graphics, vol. 21, no. 4, pp. 807–832, 2002.
- [8] S. G. Akl and G. T. Toussaint, "A fast convex hull algorithm," Information processing letters, vol. 7, no. 5, pp. 219–222, 1978.
- [9] W. F. Eddy, "A new convex hull algorithm for planar sets," // ACM Transactions on Mathematical Software, vol. 3, no. 4, pp. 398–403, 1977.
- [10] International Standard ISO/IEC 14882:2017. Programming Language C++. ISO/IEC, 2017.
- [11] Documentation CGAL 4.12 - 2D Convex Hulls and Extreme Points, 2018. [Online]. Available: https://doc.cgal.org/latest/Convex_hull_2/index.html
- [12] Chan's algorithm. [Online]. Available: https://en.wikipedia.org/wiki/Chan%27s_algorithm
- [13] B. S. Morse, Lecture 9: shape description (regions). Brigham Young University, 2000.
- [14] R. L. Graham, "An efficient algorithm for determining the convex hull of a finite planar set," Information processing letters, vol. 1, no. 4, pp. 132–133, 1972.
- [15] A. Bykat, "Convex hull of a finite set of points in two dimensions," Information Processing Letters, vol. 7, no. 6, pp. 296–298, 1978.
- [16] A. M. Andrew, "Another efficient algorithm for convex hulls in two dimensions," Information Processing Letters, vol. 9, no. 5, pp. 216–219, 1979.
- [17] J. L. Bentley, "Multidimensional binary search trees used for associative searching," Communications of the ACM, vol. 18, no. 9, pp. 509–517, 1975.
- [18] R. A. Jarvis, "On the identification of the convex hull of a finite set of points in the plane," Information processing letters, vol. 2, no. 1, pp. 18–21, 1973.
- [19] T. M. Chan, "Optimal output-sensitive convex hull algorithms in two and three dimensions," Discrete & Computational Geometry, vol. 16, no. 4, pp. 361–368, 1996.
- [20] D. G. Kirkpatrick and R. Seidel, "The ultimate planar convex hull algorithm," SIAM journal on computing, vol. 15, no. 1, pp. 287–299, 1986.